Toy IR System

Corpus

ID	Document
d_1	The star in our solar system provides heat and light.
d_2	That Hollywood star walked the red carpet for the movie premiere.
d_3	Astronomers observe distant stars and galaxies using telescopes.

Stopwords (as specified in the problem statement): {the, in, our, and, that, for}

Note on preprocessing choices. Per the problem statement, Part 1 involves *only* tokenization and stopword removal — no stemming or lemmatization. Consequently, the surface forms star (d_1, d_2) and stars (d_3) are treated as **different terms**.

Task 1: Preprocessing and Inverted Index

Tokenization + Stopword Removal

Doc	After tokenization	After stopword removal
d_1	the, star, in, our, solar, system, provides, heat, and, light	**star, solar, system, provides, heat, light**
d_2	that, hollywood, star, walked, the, red, carpet, for, the, movie, premiere	**hollywood, star, walked, red, carpet, movie, premiere**
d_3	astronomers, observe, distant, stars, and, galaxies, using, telescopes	**astronomers, observe, distant, stars, galaxies, using, telescopes**

Inverted Index

Each entry is term → [(docID, term_frequency), ...]:

Term	Postings
astronomers	[(3, 1)]
carpet	[(2, 1)]

Term	Postings
distant	[(3, 1)]
galaxies	[(3, 1)]
heat	[(1, 1)]
hollywood	[(2, 1)]
light	[(1, 1)]
movie	[(2, 1)]
observe	[(3, 1)]
premiere	[(2, 1)]
provides	[(1, 1)]
red	[(2, 1)]
solar	[(1, 1)]
star	**[(1, 1), (2, 1)]**
stars	[(3, 1)]
system	[(1, 1)]
telescopes	[(3, 1)]
using	[(3, 1)]
walked	[(2, 1)]

Only the term star is shared across multiple documents (d_1 and d_2). The plural stars in d_3 is stored as a separate entry, which will be important later.

Task 2: TF–IDF Term–Document Matrix

Weighting scheme (as implemented in InformationRetrieval):

- **TF weight** (log-normalised):

$$\mathrm{tf}_{\mathrm{weight}}(t,d);=;\{1+\log_{10}\ \mathrm{tf}(t,d)\quad \text{if } \mathrm{tf}(t,d)>0\ 0\quad \text{otherwise}$$

- **IDF** (with $N = 3$):

$$\mathrm{idf}(t);=;\log_{10}!\left(\frac{N}{\mathrm{df}(t)}\right)$$

- **TF–IDF**:

$$w_{t,d};=;\mathrm{tf}_{\mathrm{weight}}(t,d),\times,\mathrm{idf}(t)$$

Raw Term Frequencies (TF)

Term	**d₁**	**d₂**	**d₃**
astronomers	0	0	1
carpet	0	1	0
distant	0	0	1
galaxies	0	0	1
heat	1	0	0
hollywood	0	1	0
light	1	0	0
movie	0	1	0
observe	0	0	1
premiere	0	1	0
provides	1	0	0
red	0	1	0

Term	d_1	d_2	d_3
solar	1	0	0
star	1	1	0
stars	0	0	1
system	1	0	0
telescopes	0	0	1
using	0	0	1
walked	0	1	0

IDF

Term	df	N/df	$\text{idf}(t) = \log_{10}(N/\text{df})$
star	2	1.5	**0.1761**
(all other terms)	1	3.0	**0.4771**

The term star has a lower IDF because it appears in two documents; every other term appears in exactly one document and is therefore maximally discriminative.

TF–IDF Weights

Since every raw tf in this toy corpus equals 1, we have $1 + \log_{10}(1) = 1$, so each TF–IDF weight collapses to the IDF of the term:

$$w_{t,d}; =; (1 + \log_{10} 1) \cdot \text{idf}(t); =; \text{idf}(t) \text{whenever } \text{tf}(t, d) = 1.$$

Term	d_1	d_2	d_3
astronomers	0.0000	0.0000	0.4771
carpet	0.0000	0.4771	0.0000
distant	0.0000	0.0000	0.4771

Term	d_1	d_2	d_3
galaxies	0.0000	0.0000	0.4771
heat	0.4771	0.0000	0.0000
hollywood	0.0000	0.4771	0.0000
light	0.4771	0.0000	0.0000
movie	0.0000	0.4771	0.0000
observe	0.0000	0.0000	0.4771
premiere	0.0000	0.4771	0.0000
provides	0.4771	0.0000	0.0000
red	0.0000	0.4771	0.0000
solar	0.4771	0.0000	0.0000
star	**0.1761**	**0.1761**	**0.0000**
stars	0.0000	0.0000	0.4771
system	0.4771	0.0000	0.0000
telescopes	0.0000	0.0000	0.4771
using	0.0000	0.0000	0.4771
walked	0.0000	0.4771	0.0000

Document vector norms:

Doc	$\lVert\cdot\rVert_2$
d_1	1.0813
d_2	1.1819

Doc	$\|\cdot\|_2$
d_3	1.2623

Task 3: Boolean Retrieval — Query "star light"

Query processing

- Tokens after stopword removal: [star, light]

Inverted-index lookup

Term	Docs containing it
star	{1, 2}
light	{1}

AND intersection

Documents containing **all** query terms:

$$1,2 \cap 1 = 1$$

Boolean result: [d_1]

Only d_1 contains both terms. d_2 contains star but lacks light; d_3 contains neither (it has stars, not star, which is not matched without stemming).

Task 4: Cosine Similarity Ranking — Query "star light"

Query TF-IDF vector

Term	TF	$1+\log_{10}(tf)$	IDF	TF-IDF
star	1	1	0.1761	0.1761
light	1	1	0.4771	0.4771

Query norm = $\sqrt{(0.1761^2 + 0.4771^2)}$ = **0.5086**

Cosine similarities

$$\cos(q, d) = \frac{q \cdot d}{|q| \cdot |d|}$$

Doc	Dot product	‖d‖	Cosine similarity
d_1	0.1761·0.1761 + 0.4771·0.4771 = 0.2587	1.0813	**0.4703**
d_2	0.1761·0.1761 = 0.0310	1.1819	**0.0516**
d_3	0	1.2623	**0.0000**

Final ranking

$$\boxed{d_1 \succ d_2 \succ d_3}$$

Is the ranking desirable?

Yes — the top-1 choice (d_1) is clearly correct. A user asking for "star light" almost certainly means astronomical star-light. d_1 (*"The star in our solar system provides heat and light."*) is the only document genuinely on topic.

However, the ranking exposes two limitations even in this toy example:

1. **d_2 scores non-zero purely because of polysemy.** "Star" in d_2 means *celebrity*, but cosine similarity has no way to know that — it just sees a token match and awards 0.0516. In a larger corpus this spurious contribution accumulates and can push irrelevant docs above relevant ones.
2. **d_3 scores exactly 0 despite being topically relevant** to "stars".
   Because star and stars are different surface forms under this preprocessing, the VSM completely misses the connection. This is the classic morphological mismatch problem that stemming/lemmatization is intended to solve.

So the ranking is **desirable in direction but not in magnitude**: the correct document is first, yet the scores understate d_3 (should be positive) and overstate d_2 (should be ~0).

Task 5: Word Sense Ambiguity — Query "movie star"

Ideal behaviour

The phrase "movie star" unambiguously refers to a *film celebrity*. The only document matching that sense is **d_2** (*"That Hollywood star walked the red carpet…"*). d_1 uses "star" in an astronomical sense; d_3 doesn't contain the word "movie" at all.

What the system actually returns

Query tokens: [movie, star]

Query TF-IDF: movie: 0.4771, star: 0.1761, norm = 0.5086

Doc	Dot product	Cosine similarity
d_1	$0.1761 \cdot 0.1761 = 0.0310$	**0.0564**
d_2	$0.4771 \cdot 0.4771 + 0.1761 \cdot 0.1761 = 0.2587$	**0.4303**
d_3	0	**0.0000**

Ranking: [d_2, d_1, d_3] — d_2 is correctly on top.

How word sense ambiguity affects retrieval

Although d_2 ranks first, d_1 receives a **non-zero similarity of 0.0564 purely from the shared token star**. This is incorrect semantically: d_1's "star" is a celestial object, not a celebrity. The Vector Space Model represents every occurrence of star with the same vector coordinate, collapsing both senses into one dimension.

Concretely, polysemy hurts retrieval in three ways here:

1. **Spurious matches**: d_1 is awarded credit for a word it only *syntactically* shares with the query.
2. **Score inflation of irrelevant docs**: in a larger corpus, many documents mentioning astronomical stars would all get partial credit for "movie star" queries.
3. **Missed relevant docs**: conversely, documents about film celebrities that happen to use synonyms ("actor", "actress", "celebrity") instead of "star" would score zero — the same word-level rigidity works against recall.

These observations motivate the Part-5 improvements to the IR system. Possible remedies already covered in the course notes include:

- **Lemmatisation / stemming** to unify star/stars and partially mitigate morphological mismatch.
- **Latent Semantic Analysis (LSA)** — SVD on the term–document matrix projects both senses of "star" into a latent space where co-occurring context (solar, heat, light vs. hollywood, carpet, premiere) pulls d_1 and d_2 apart along different latent dimensions, reducing cross-sense interference.
- **Explicit Semantic Analysis (ESA)** — represents documents as weighted vectors over Wikipedia concepts, providing external world knowledge that naturally separates the celestial vs. celebrity senses of star.
- **Word Sense Disambiguation (WSD)** during indexing — tag each occurrence of star with its intended sense before building the index.

Summary Table

Task	Query	Result	Correct?
Boolean retrieval	"star light"	$[d_1]$	✓
Cosine ranking	"star light"	$d_1 \succ d_2 \succ d_3$	✓ (top-1), but d_3 undervalued due to star/stars mismatch
Cosine ranking	"movie star"	$d_2 \succ d_1 \succ d_3$	✓ (top-1), but d_1 gets spurious credit due to polysemy of star